



Objektno orijentirano programiranje

Jesenski ispitni rok

31.8.2021.

Ispit nosi ukupno 50 bodova i piše se 150 minuta.

U zadacima nije potrebno pisati dio u kojem se uključuju klase ili paketi klasa (import)

Rješenja je potrebno pisati na stranice ispita. Ako na nekoj stranici ne budete imali dovoljno mjesta, ostatak možete napisati na zadnju (praznu) stranicu ispita te isto naznačiti na zadatku kojeg nadopunjujete.

IZJAVA

Tijekom ove provjere znanja neću od drugoga primiti niti drugome pružiti pomoć te se neću koristiti nedopuštenim sredstvima.

Ove su radnje povreda Kodeksa ponašanja te mogu uzrokovati trajno isključenje s Fakulteta.

Zdravstveno stanje dozvoljava mi pisanje ovog ispita.

Vlastoručni potpis studenta: _____

1. zadatak (10 bodova)

Potrebno je modelirati sustav vjernosti kupaca u trgovačkom lancu. **Vaš zadatak je nadopuniti predložak UML dijagrama klasa ovog sustava koji se nalazi na sljedećoj stranici.** Pritom **možete dodati nove klase** (ili sučelja) ako smatrate da je to potrebno. **Nije potrebno pisati programski kod sustava i to neće donositi nikakve bodove.**

Na ovom predlošku pažljivo navedite:

- oznake za sučelje (**I**), enumeraciju (**E**), apstraktnu (**A**) ili običnu klasu (**C**)
- oznake za apstraktnu metodu (**A**) te oznake za statičku (**S**) i finalnu (**F**) metodu ili atribut
- modifikatore vidljivosti metoda i atributa (+ public, # protected, - private, ~ package-private)
- tipove povratnih vrijednosti i argumenata za metode te tipove atributa
- odgovarajuće strelice da naznačite nasljeđivanje klasa (puna linija $\rightarrow$), implementaciju sučelja (iscrtkana linija $\dashrightarrow$) ili asocijaciju (puna linija, prazna strelica $\longrightarrow$)

Trgovački lanac bilježi svoje proizvode (**Product**). Za svaki proizvod prati se njegova šifra (**id**, String), naziv (**name**, String), cijena (**price**, double), te broj bodova vjernosti koji se skupi kupnjom tog proizvoda (**points**, int). Trgovački lanac također ima programe vjernosti (**LoyaltyProgram**) za kupce. Svaki program sadržava podatak o šifri programa (**id**, String), naziv programa (**name**, String), količinu bodova potrebnih za svakih 5% popusta (**ptsFivePercentDiscount**, int), te maksimalni mogući postotak popusta za taj program (**maxDiscount**, int). Svaki program nudi i listu proizvoda (**awards**, Set), iz koje kupac može odabrati jedan proizvod kao nagradu kod trošenja bodova vjernosti. Program vjernosti ne mora imati nagradu. Program vjernosti ima metodu za dodavanje proizvoda (**Product**) u listu nagrada (**addAward**).

Bilježe se podaci o kupcima (**Customer**), koji sadrže šifru kupca (**id**, String), osobni identifikator (**OIB**, String), ime (**name**, String), prezime (**surname**, String), datum rođenja (**dateOfBirth**, Date). Kupac može imati točno jedan račun vjernosti (**account**, **LoyaltyAccount**), koji se dodaje kroz metodu **addAccount**. Svaki račun vjernosti pripada točno jednom programu vjernosti (**program**, **LoyaltyProgram**), dok svaki program vjernosti sadrži listu svih pripadajućih računa vjernosti (**accounts**, Set). Za svaki račun vjernosti bilježimo njegov broj (**accountNo**, String) i trenutni broj bodova na računu (**points**, int).

Program vjernosti (**LoyaltyProgram**) ima metodu za stvaranje novog računa vjernosti (**createAccount**), koji prima kao parametre broj računa (String) i kupca (Customer).

Kupcima se za kupovinu izdaju kartice vjernosti (**LoyaltyCard**). Svaka kartica vjernosti ima točno jednog kupca. Za karticu vjernosti bilježi njen broj (**cardNo**, String), te lista transakcija po toj kartici (**transactions**, Transaction). Prilikom kupovine, kupac blagajniku daje karticu vjernosti, nakon čega ga blagajnik obavještava o broju bodova na računu vjernosti, te popustu koji ostvaruje u programu vjernosti. Svaka kupovina predstavlja transakciju (**Transaction**), koja ovisno o tome da li kupac želi trošiti bodove može biti **SpendTransaction** ili **EarnTransaction** ako kupac želi nastaviti skupljati bodove. Svaka transakcija ima svoju šifru (**id**, int), datum (**date**, Date), broj bodova promjene (**points**, int), te listu od barem jednog kupljenog proizvoda (**items**, List<Product>). **SpendTransaction** dodatno bilježi ukupni ostvareni postotak popusta (**discount**, int), te nagradu koju je kupac odabrao (**award**, Product) ukoliko program vjernosti ima nagrada. Kartica vjernosti ima metodu za dodavanje transakcije kojom se skuplja bodove (**addEarnTransaction**), a koja prima listu kupljenih proizvoda (List<Product>), te metodu za dodavanje transakcije kojom se bodovi troše (**addSpendTransaction**), koja prima listu kupljenih proizvoda (List<Product>) i odabranu nagradu (Product).

Račun vjernosti (**LoyaltyAccount**) ima metodu za ažuriranje bodova **updatePoints** koja prima transakciju (**Transaction**) kao parametar.

--

LoyaltyProgram

LoyaltyAccount

Customer

Product

Transaction

LoyaltyCard

SpendTransaction

EarnTransaction

2. zadatak (10 bodova)

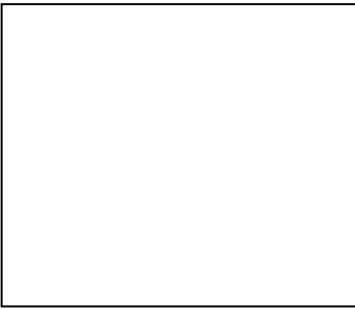
Potrebno je implementirati klasu `ListTypeAnalyzer` koja nasljeđuje `SimpleFileVisitor<Path>` i to na način da njena javna metoda `public Map<String, Long> getSizesByExtension()` po izvođenju metode `Files.walkFileTree` ostvaruje sljedeću zadaću:

metoda treba vratiti mapu u kojoj su **ključevi ekstenzije iz liste ekstenzija za koju je klasa bila instancirana**, a **vrijednost je zbirni iznos veličina svih pronađenih datoteka koje imaju dotičnu ekstenziju** gledano od početnog direktorija i dublje (početni direktorij se zadaje u pozivu `walkFileTree` metode). Datoteke mogu, ali ne moraju sadržavati ekstenziju.

Dodatno, po izvođenju metode `Files.walkFileTree` potrebno je:

- Za svaki direktorij izračunati **ukupnu veličinu** svih **datoteka zadanih ekstenzija zbirno (ali ne i njegovih poddirektorija)** koje se nalaze u tom direktoriju.
- Veličinu direktorija potrebno je ispisivati na standardni izlaz počevši od dna stabla prema vrhu.

```
public class ListTypeAnalyzer extends SimpleFileVisitor<Path>    {
```



```
@Override  
public FileVisitResult visitFile(Path file, BasicFileAttributes attrs) throws IOException {
```

```
}
```

```
@Override  
public FileVisitResult postVisitDirectory(Path dir, IOException exc) throws IOException {
```

```
}
```

```
}
```

3. zadatak (10 bodova)

Zadani su razredi `ResourceException`, `ResourceProto` i `Resource` koji implementira sučelje `Closeable`. Što će program ispisati tijekom prevođenja i izvođenja metode `main`? (u slučaju da program nije izvediv, napišite da se tijekom prevođenja pronalazi pogreška). **Rješenje upisati u iscrtkani okvir na ovom ispitu.**

```
public class ResourceException extends RuntimeException {}

public class ResourceProto {
    protected int p;

    public ResourceProto(int p) {
        this.p = p;
        System.out.println(p + " created");
    }

    public void close() throws IOException {
        System.out.println(p + " closed in Proto");
    }

    public void work() throws ResourceException {
        if (p == 2)
            throw new ResourceException();
        System.out.println(p + " work ends " + p);
    }
}

public class Resource extends ResourceProto implements Closeable {

    public Resource(int p) {
        super(p);
    }

    public static void main(String[] args) {

        List<Resource> resources = new ArrayList<>();
        resources.add(new Resource(1));
        resources.add(new Resource(2));
        resources.add(new Resource(3));

        for (Resource r : resources) {
            try (r) {
                r.work();
            } catch (ResourceException e) {
                System.out.println("Resource Exception");
            } catch (IOException e) {
                System.out.println("IO Exception");
            } catch (RuntimeException e) {
                System.out.println("Runtime Exception");
            }
        }

        try (Resource r3 = new Resource(9); Resource r1 = new Resource(-1);) {
            Resource r0 = new Resource(0);
            r3.work();
            r1.work();
            r0.work();
            System.out.println("try ends");
        } catch (Exception e) {
            System.out.println("exception");
            return;
        } finally {
            System.out.println("finally");
        }
        System.out.println("end");
    }
}
```

4. zadatak (10 bodova)

Program za evidenciju posudbi u školskoj knjižnici podatke iz baze podataka učitava u mapu **map** (spremnik tipa **Map<K,V>**), i to tako da kao ključ jednog zapisa sprema informacije o primjerku knjige, modelirane klasom **Book**, a kao vrijednost **listu** njenih posudbi (svaki element liste je objekt klase **Borrowing**). Knjiga je opisana jedinstvenim serijskim brojem **ISBN** (Integer), naslovom **title** (String), imenom autora **author** (String) i žanrom **genre**

(String), dok je svaka posudba opisana datumima posudbe i vraćanja **borrowStart** i **borrowEnd** (LocalDate, ako knjiga nije vraćena, datum vraćanja je null) te imenom osobe koja je knjigu posudila, **pupil** (String). Pretpostavite da u razredima Book i Borrowing postoje svi getteri i setteri.

Npr. ovako bi mogao izgledati dio zapisa u mapi:

K		V
255,"Vodič kroz galaksiju za autostopere","Douglas Adams","SF"	->	[(2021-01-03,2021-02-01,"Ana Anić"), (2021-06-23,2021-07-23,"Ivo Ivić")]
101,"Trojica u Trnju","Pavao Pavličić","CRIME"	->	[(2020-12-02,2020-12-30,"Mirko Mirić"), (2021-05-03,2021-06-02,"Ivo Ivić"), (2021-08-25,null,"Pero Perić")]
761,"The colour of magic","Terry Pratchett","FANT"	->	[(2021-08-25,null,"Sara Sarić")]
799,"Pavao Pavličić","Petlja","KRIM"	->	[(2021-08-25,null,"Sara Sarić")]
935,"Gola istina","Nives Celzijus","BIO"	->	[] //prazna lista

Koristeći naredbe iz Java Stream API-ja nadopunite sljedeći programski kod:

a) vraća broj knjiga koje spadaju u žanr SF (1 bod)

```
long countSF = map.
```

b) ispisuje naslove svih knjiga čiji autor je Pavao Pavličić, poredane abecedno od A do Z (2 boda)

```
map.
```

c) vraća novu mapu kojoj je ključ šifra knjige, a vrijednost broj koliko puta je knjiga posuđena (3 boda)

```
Map<Integer, Integer> newMap = map.
```

d) vraća listu svih knjiga koje trenutno posuđuje Sara Sarić (4 boda)

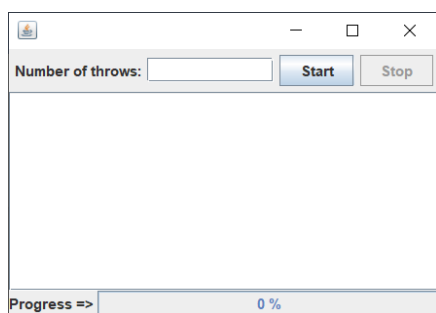
```
List<Book> pknjige = map.
```

5. zadatak (10 bodova)

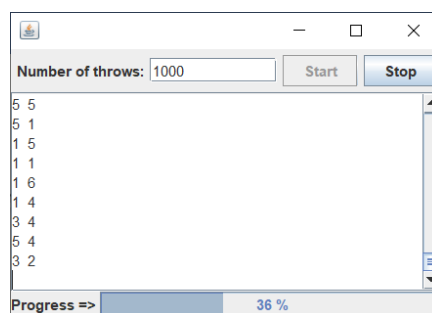
Grafička aplikacija omogućava bacanje dvije kocke pri čemu se želi simulacijom odrediti vjerojatnost događaja u kojem je zbroj vrijednosti kocaka jednak 6. Aplikacija je predstavljena klasom GUI koja u sebi ima definirane dvije unutrašnje klase InputPanel i ThrowingDiceTask. InputPanel predstavlja dio sučelja putem kojeg se može upisati željeni broj bacanja te pokrenuti i zaustaviti bacanja. Budući da su bacanja vremenski zahtjevan posao, on se izvodi u sklopu klase ThrowingDiceTask koja je

SwingWorker. Vaš je zadatak da:

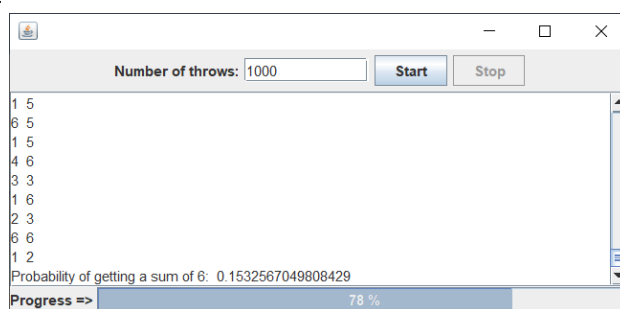
- U konstruktoru klase GUI dodate i rasporedite komponente grafičkog sučelja na način da ono izgleda kao na slici 1 koja prikazuje izgled sučelja prilikom pokretanja aplikacije (na slici 3 imate prozor prikazan nakon proširivanja)
- U konstruktoru klase InputPanel definirajte osluškivače gumba Start i Stop. Koristite lambda izraze. Slika 2 prikazuje sučelje nakon pritiska gumba Start dok su bacanja u tijeku pri čemu je broj bacanja bio upisan pravilno - kao cijeli broj. Tada se briše (stari) rezultat bacanja, gumbi se postavljaju u odgovarajuće stanje te se pokreću bacanja. Ako u polje za upis željenog broja bacanja nije upisan broj u odgovarajućem formatu onda se samo izbriše sadržaj tog polja i ne radi se ništa drugo. Kod pritiska na gumb Stop, samo se postavlja zastavica `isStopped` tako da se prekine izvođenje bacanja (sl 3).
- Dopunite zaglavlje klase ThrowingDiceTask i dopunite kod metode `doInBackground` na način da se omogući ispis rezultata svakog bacanja (par vrijednosti, vidi sliku 2 ili 3) i naravno da se osigura pomicanje pokazivača napretka. U skladu s tim trebate dopuniti metodu `process`. Završetak bacanja nastupa ili kad se pritisne gumb Stop (sl. 3) ili kad se postigne željeni broj bacanja. U tom trenutku se u polju rezultata ispisuje "Probability of getting a sum of 6:" iza čega slijedi vrijednost omjera pojave relevantnog događaja i ukupnog broja bacanja i naravno resetiraju se stanja gumba i zastavice.



1

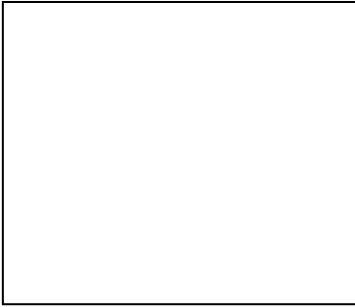


2



3

```
public class GUI extends JFrame {
    private JTextArea result = new JTextArea();
    private JProgressBar progressBar = new JProgressBar(0, 100);
    private JTextField noOfThrows = new JTextField(10);
    private JButton startButton = new JButton("Start");
    private JButton stopButton = new JButton("Stop");
    private JLabel progress = new JLabel("Progress ==> ");
    private boolean isStopped = false;
```

```
public GUI(){
    result.setRows(10);
    progressBar.setStringPainted(true);
    //TODO - add progress label and progress bar
```

```
    //TODO - add input panel and result text area with scroll bars
```

```
} // end of GUI constructor
```

```
private class InputPanel extends JPanel {
    InputPanel() {
        add(new JLabel("Number of throws:") );
        add(noOfThrows);
        add(startButton);
        add(stopButton);
        stopButton.setEnabled(false);
        //TODO - define action on Start click
```

```
    //TODO - define action on Stop click
```

```
    }// end of InputPanel constructor
} // end of InputPanel
```

```
private class ThrowingDiceTask _____ >
{ //TODO - complete hader
```

```
    private final int noThrows;
    ThrowingDiceTask(int noThrows) {
        this.noThrows = noThrows;
    }
}
```

```
@Override public Double doInBackground() {
    int counter = 0, counterSix = 0, dice1, dice2;
    while (( counter < this.noThrows ) && ! isStopped) {
        dice1 = (int)(Math.random()*(6))+1;
        dice2 = (int)(Math.random()*(6))+1;
        if((dice1+dice2)==6) counterSix++;
        //TODO - add necessary code
    }
}
```

```
        try {
            Thread.sleep(10);
        } catch (InterruptedException e) {}
    }
    return _____ ; //TODO - add necessary code
}
```

```
@Override protected void process(List<String> chunks) {
    //TODO - add necessary code //
}
```

```

}

@Override protected void done() {
    //TODO - add necessary code
}
```

```
    }
} // end of ThrowingDiceTask
```

```
public static void main(String[] args) {
    // ...
}
```

```
} // end of GUI
```

